Deep Learning Video Object Tracking

CS-5567 Deep Learning

Group-5

Team Members

Reethu Gopavarapu

- M.S. in Data Science
- B.Tech in Information Technology
- Interested in Data Analytics and Data Science

Sahithi Thota

- M.S. in Computer Science
- B.E. in CS Engineering
- Interested in Artificial Intelligence and Data Science

Hyma Reddy

- M.S. in Computer Science
- B.Tech in CS Engineering
- Interested in Machine Learning and Autonomous Systems

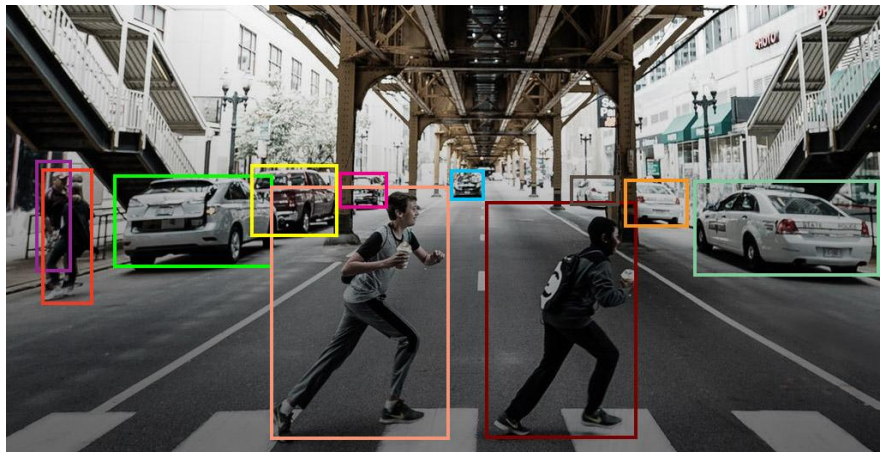
Project Overview & Objectives

Overview:

This project focuses on developing a deep-learning-based pipeline for detecting and tracking multiple moving objects across video frames using the **MOT16 dataset**. It combines **Faster R-CNN** for object detection and a **Siamese Network** for re-identification to maintain consistent object identities over time.

Objectives:

1.Prepare and augment the MOT16 dataset for robust training.
2.Fine-tune Faster R-CNN for high-accuracy object detection.
3.Implement a Siamese Network for object re-identification.
4.Integrate detection + Re-ID for a complete tracking pipeline.
5.Evaluate system accuracy and visualize tracking outputs.



Project Timeline and Team Contribution

Dataset Preparation

Week- 1 focused on literature review and preparing the MOT16 dataset, led by Hyma

Re-Identification

Week- 3 involved implementing a Siamese Network for object re-identification by Sahithi

Evaluation

Week- 5
Involved evaluation, visualization, and presentation preparation by all group

Detection

Week- 2 dedicated to object detection using Faster R-CNN, managed by Reethu

Tracking

Week- 4 centered on object association and tracking, handled by Sahithi

Methodology & Workflow

Dataset Preparation

Collect and preprocess the MOT16 dataset with augmentations such as color jitter, blur, and normalization.

Object Detection

Use a Siamese Network to match object identities consistently across frames.

Re-Identification

Use Siamese Network to match object identities consistently across frames.

Object Association

Combine detection and Re-ID embeddings to maintain continuous tracking IDs.

Evaluation & Visualization

Measure tracking metrics like MOTA and IDF1, and generate annotated videos for result analysis.

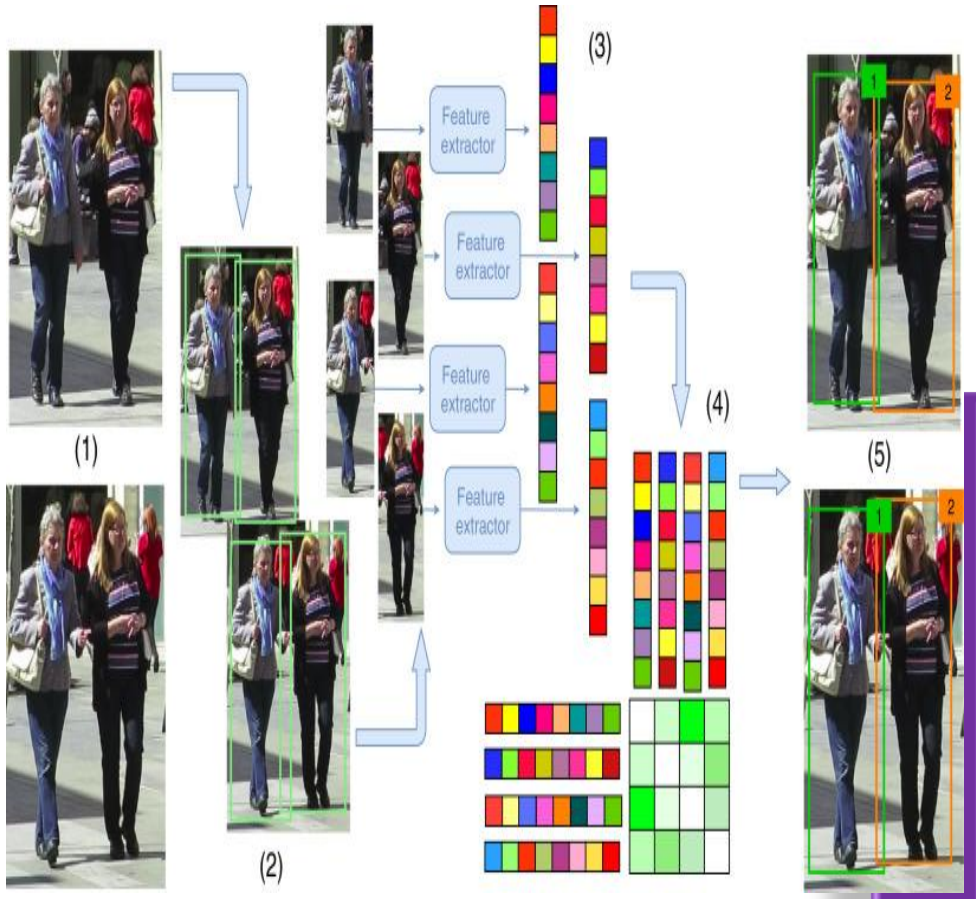
Model Overview

Faster R-CNN:

• Role: Object detection within each frame.
• Components: Feature Extractor, Region Proposal Network, ROI Pooling, Classifier.

Siamese Network:

• Role: Re-identification & identity preservation.
• Components: Feature Extractor (ResNet-18), Similarity Metric (L2 distance), Contrastive Loss.



Model Integration:

• Combined use of Faster R-CNN and Siamese Network enables detection and continuous tracking of multiple objects in video sequences.

Dataset Description & Technical Approach

Dataset: MOT16 Benchmark

•14 video sequences (7 train + 7 test)
•High-resolution pedestrian scenes
•Each frame annotated with bounding boxes and IDs

Technical Approach:

•Platform: Google Colab (T4 GPU)
•Libraries: PyTorch, Torchvision, OpenCV, NumPy, Matplotlib
•Data Augmentations: Color Jitter, Gaussian Blur, Random Grayscale, Normalization

Implementation:

Data Augmentation :

Transformations (applied randomly):

- Color Jitter
- Gaussian Blur
- Motion Blur
- Grayscale
- Normalization

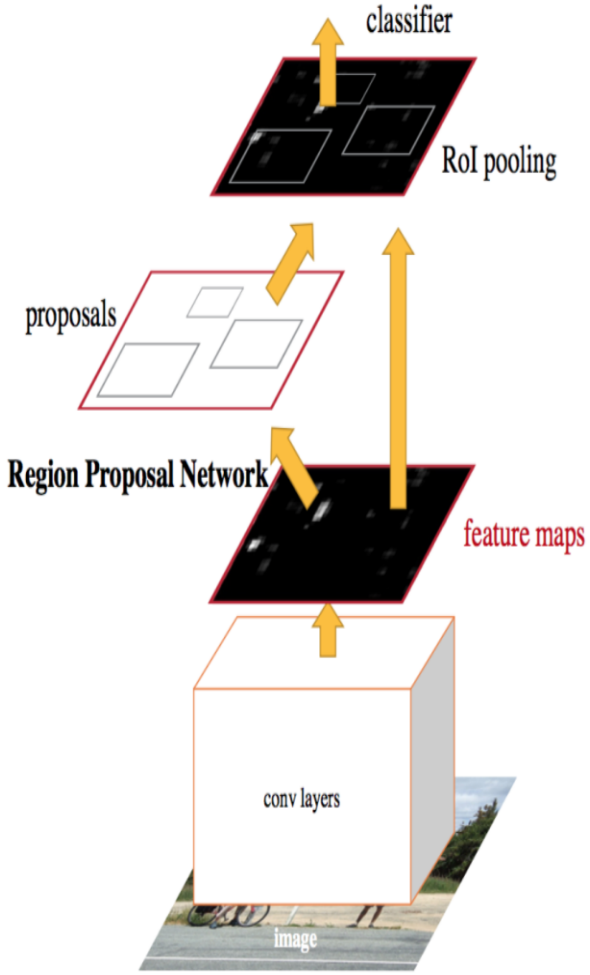
Benefits:

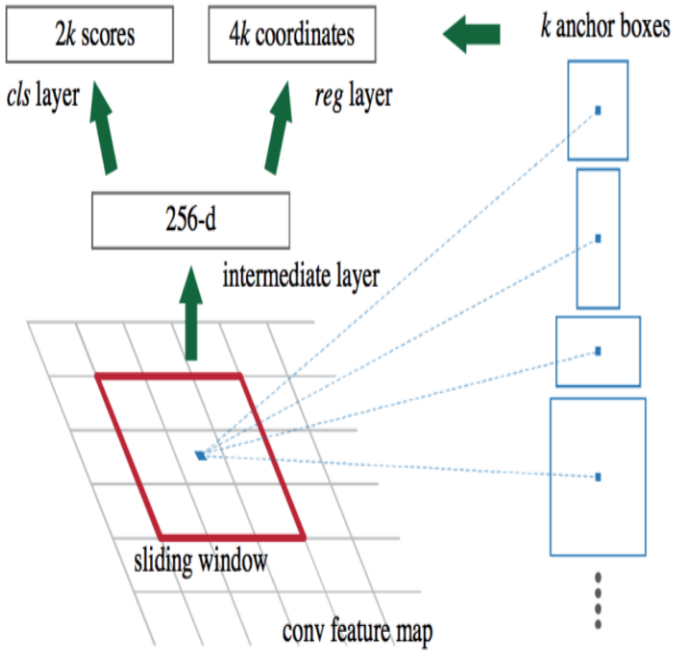
- Enhanced robustness and generalizability in both object detector and re-ID models

Faster R-CNN

- ResNet50 Backbone
- Feature Pyramid Network (RPN applied at different scales/levels of the backbone CNN

- 4 loss types
  - Classification loss
  - Objectness loss
  - Box Regression loss
  - RPN Box Regression Loss





Siamese Network:

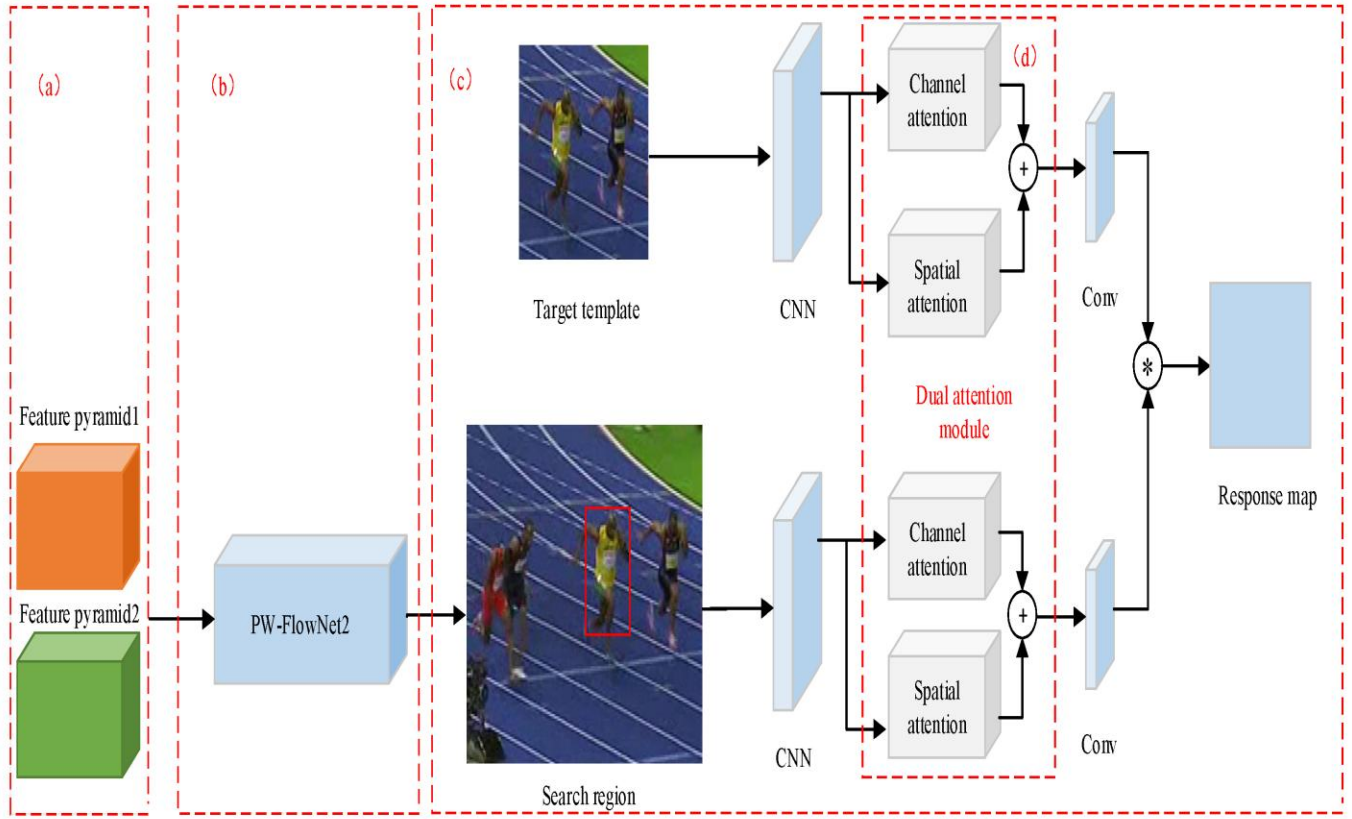
- ResNet-18 backbone (feature extraction) + linear layer (256-embedding)
- Network trained using Contrastive Loss: - Minimize for positive pairs
- Maximize for negative pairs (up to m)
- Regularized to reward compact representation

Pair Selection in Siamese Network:

- Positive Pairs: Same object
- Negative Pairs: Different objects
- Temporal proximity: Within 10 frames

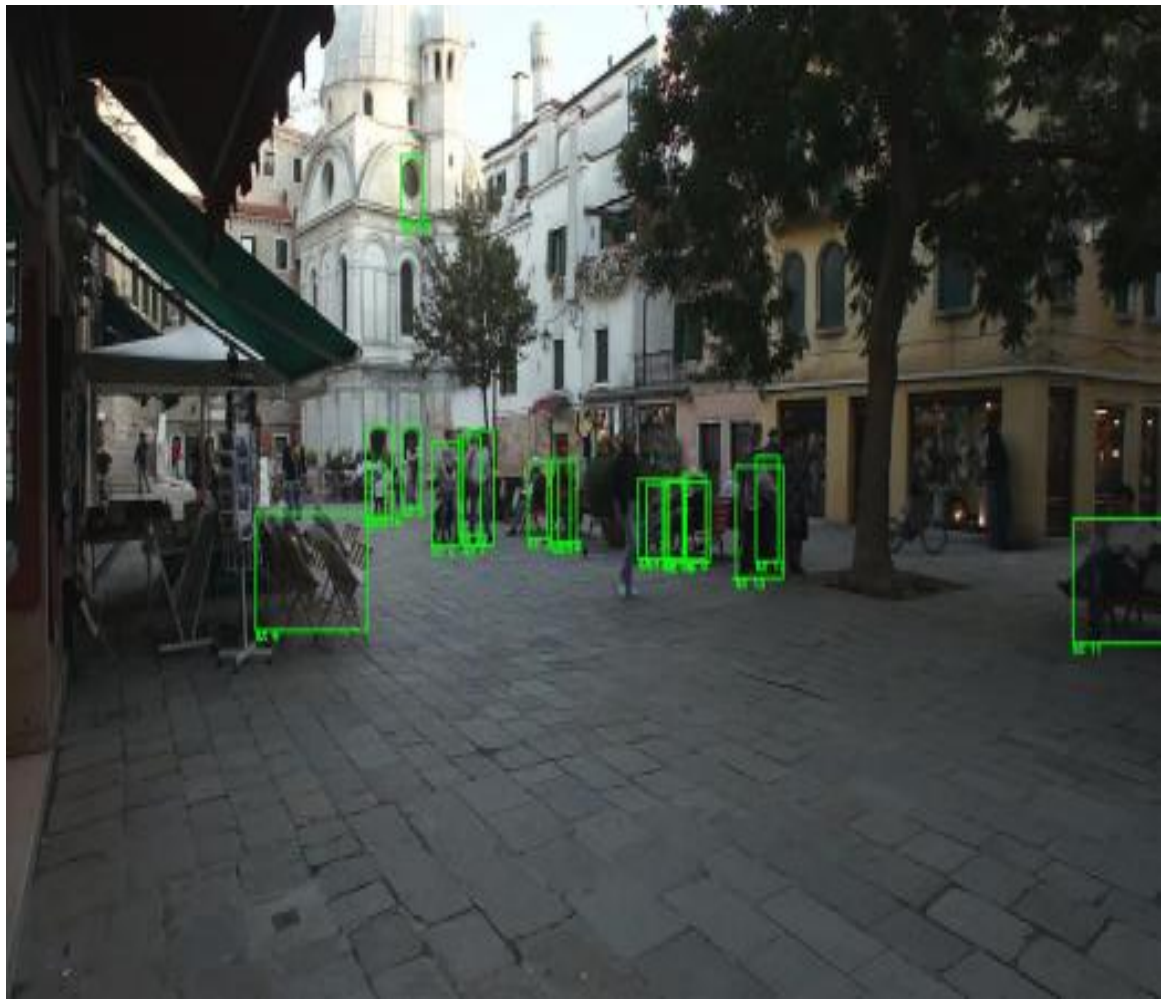
Siamese Network Training Pipeline:

- Dataset > Data Loader > Training Procedure > Evaluation > Model Persistence

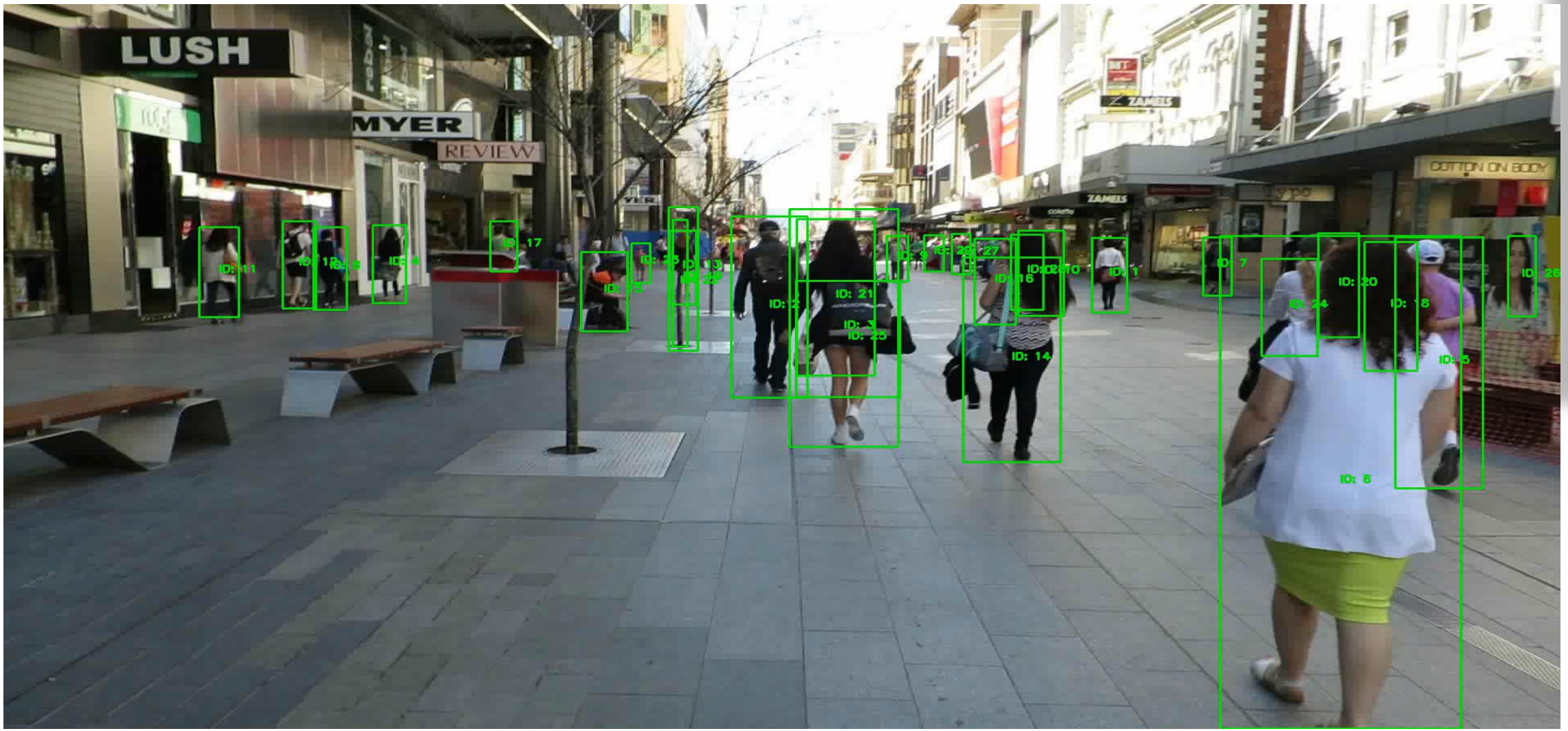


Object Tracking Pipeline

1. Load pre-trained object detection and re-ID models
2. Object detection inference -> predicted bounding boxes
3. Retain boxes with confidence > 70%
4. Re-ID inference -> embedding vectors
5. Match with tracked objects (L2 distances < 0.7)
6. Store predicted boxes and object IDs in a data frame
7. Generate a video with predictions displayed



Results:


LUSH
MYER
REVIEW
ZAMELS
ID: 11
ID: 17
ID: 21
ID: 14
ID: 20
ID: 24
ID: 18
ID: 26
ID: 8

Evaluation:

Key Findings:

- Detection loss decreased from 0.91 → 0.53 in 5 epochs.
- Re-ID loss reduced from 0.058 → 0.033 over 10 epochs.
- Generated annotated videos showing bounding boxes and consistent IDs.
- Average training time: Detection ~ 1.5 hrs; Re-ID ~ 11.5 hrs (T4 GPU).

Challenges Faced:

- **Limited GPU Access:** Colab queue delays and training interruptions.
- **Image Transform Issues:** Hard to visualize augmented tensors → needed custom functions.
- **ID Switches:** Re-ID confusion during occlusions → improved with cropped inputs.
- **Non-unique IDs:** IDs repeated across sequences → restricted within each video.
- **Training Time:** Re-ID model training took > 10 hours per run.

Future Work & Enhancements:

- Integrate **YOLOv9 + BYTE Track + BoT-SORT + OSNet** for real-time tracking.
- Explore **Transformer-based MOT models** (TrackFormer, MOTR).
- Deploy on **edge devices** (Jetson Nano, Raspberry Pi) for real-world use.
- Bounding boxes in **different colors** per object type.
- Real-time **object IDs above each box**.
- Display **object name + confidence score**(e.g., “Person 98%”).
- Optionally render **tracking trails** (path lines).

Conclusion & Key Learnings:

Conceptual Learnings:

- Understood multi-object tracking concepts, detection vs re-identification.
- Learned how data augmentation and feature embeddings impact tracking accuracy.

Technical Learnings:

- Gained hands-on experience with PyTorch for deep learning.
- Improved skills in model fine-tuning, data handling, and Colab GPU management.
- Learned evaluation metrics (MOTA, IDF1) and tracking visualization techniques.


THANK
YOU